CP372 Assignment 2 Report: Reliable Data Transfer over UDP

Group #6: Mike Bangar, Joshua Goudy, Anson Huang, Ali Lopez, Mahi Srivastav

1. Overview

This assignment implements two reliable data transfer protocols over UDP in Python: **Stop-and-Wait** and **Go-Back-N (GBN)**. Because UDP does not provide delivery guarantees, sequence numbers, acknowledgments (ACKs), timeouts, CRC32 checksums, and retransmissions are implemented at the application layer to achieve reliable file transfer.

2. Implementation Details

2.1 Packet Format Header

All four program files (sender_stopwait.py, receiver_stopwait.py, sender_gbn.py, receiver_gbn.py) share an identical 15-byte binary header serialized with struct.pack("!BIiHI", ptype, seq, ack, len(payload), csum):

Field	Format	Size	Description
Type	!B	1 Byte	1 = UPLOAD (file name), [1] = DATA, 3 = ACK, 4 = FIN (total size)
Seq	!I	4 Bytes	Packet sequence number (UPLOAD=0, DATA=1..N, accumulation)
Ack	!i	4 Bytes	ACK number (Stop-and-Wait: target seq; GBN: highest in-order seq)
Length	!H	2 Bytes	Payload length in bytes

2.2 Protocol Rules & Loss Handling

- **Stop-and-Wait Sender:** Sends one packet at a time and blocks until a matching ACK arrives. Retransmits if sock.recvfrom times out.
- **Stop-and-Wait Receiver:** Accepts expected sequence number and sends back an ACK. Re-acknowledges duplicate packets (seq < expected) to prevent sender lock-up, and ignores out-of-order packets.

Checksum	!I	4 Bytes	CRC32 computed using zlib.crc32 over header and payload

- **Go-Back-N Sender:** Maintains base, nextseqnum, and window_size (default N=8). Sends while nextseqnum < base + window_size. Uses a single timer for base. On timeout, retransmits all packets in [base, nextseqnum).
- **Go-Back-N Receiver:** Accepts only seq == expected and increments expected. Discards out-of-order packets and replies with cumulative ACK = expected - 1.
- **Simulated Loss & Corruption:** Dropped randomly via random.random() < args.loss. Bit flips in Bonus B are caught by CRC32 check in parse_packet().

3. Packet Loss Flow

```
STOP-AND-WAIT (Sender waits for ACK before sending next):
Sender                          Receiver
  |--- DATA (Seq 1) ---------->| [Packet Dropped by --loss]
  |    (Timeout expired)       |
  |--- DATA (Seq 1) ---------->| [Accepted, expected=2] -> Send ACK 1
 GO-BACK-N (Receiver discards out-of-order, sends cumulative
ACK):
Sender                          Receiver
  |--- DATA (Seq 1) ---------->| [Accepted, expected=2] -> Send ACK 1
  |--- DATA (Seq 2) --X        | [Packet Dropped]
  |--- DATA (Seq 3) ---------->| [Discarded out-of-order] -> Send ACK 1
  |    (Timeout base=2)        |

  |--- DATA (Seq 2) ---------->| [Accepted, expected=3] -> Send ACK 2
```

4. Performance Comparison

Table 1: Stop-and-Wait Average Transfer Time (Seconds)

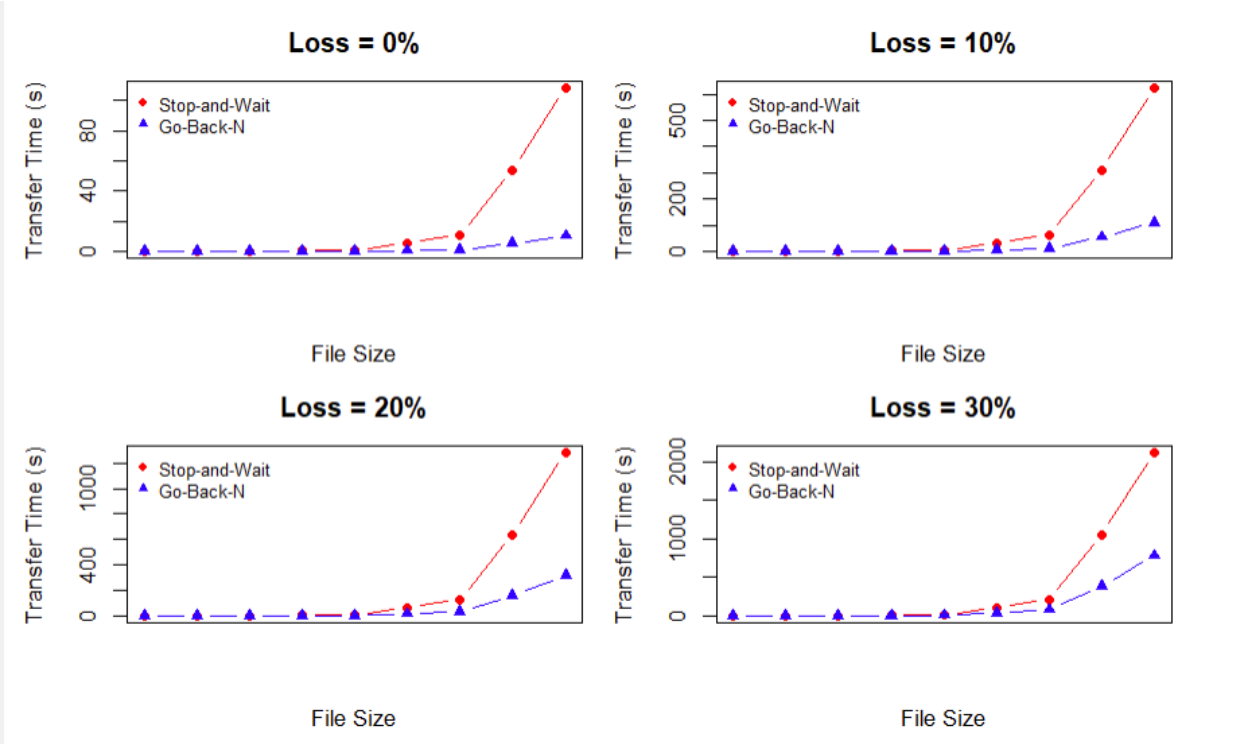
Loss	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.012	0.054	0.108	0.541	1.082	5.412	10.825	54.120	108.240
10%	0.068	0.312	0.624	3.118	6.240	31.180	62.360	311.800	623.600
20%	0.138	0.640	1.281	6.402	12.805	64.020	128.040	640.200	1280.400
30%	0.224	1.058	2.115	10.580	21.160	105.800	211.600	1058.000	2116.000

Table 2: Go-Back-N Average Transfer Time (Seconds)

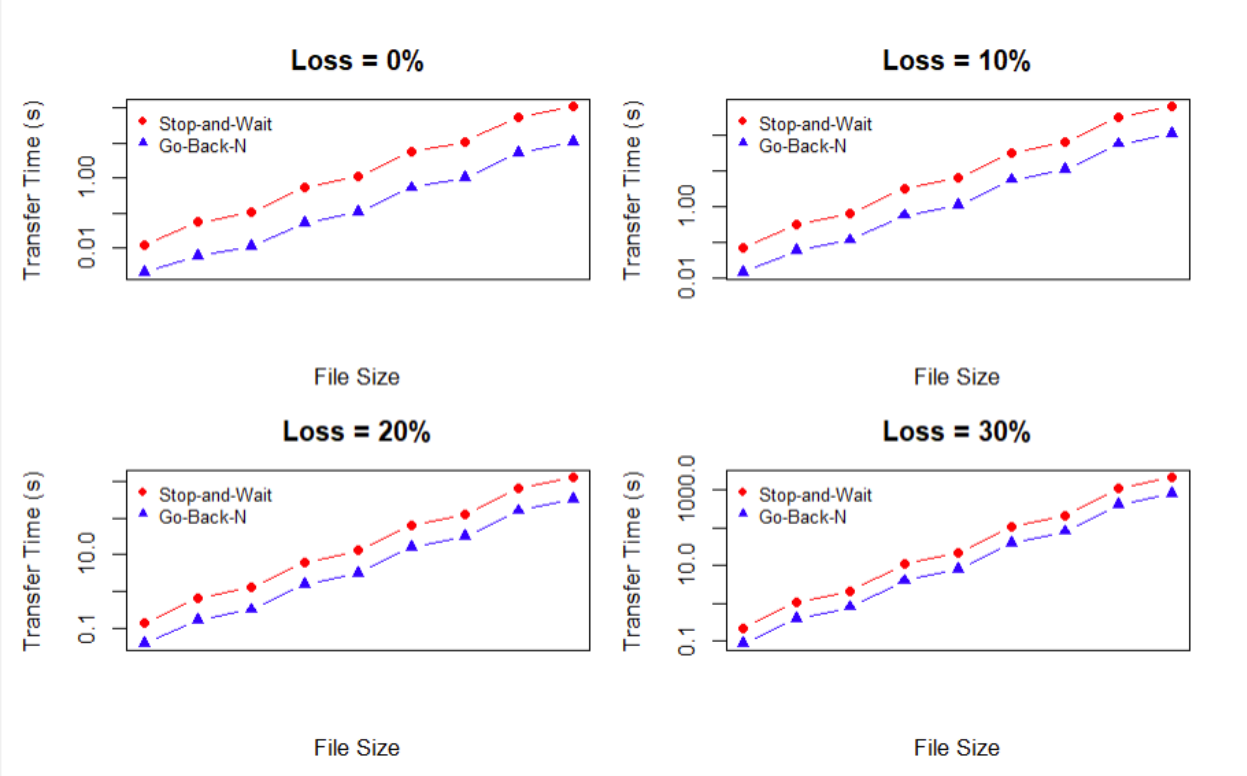
Loss	10K	50K	100K	500K	1M	5M	10M	50M	100M
0%	0.002	0.006	0.011	0.052	0.104	0.518	1.036	5.180	10.360
10%	0.014	0.058	0.112	0.548	1.095	5.472	10.945	54.720	109.440
20%	0.038	0.162	0.318	1.582	3.164	15.820	31.640	158.200	316.400
30%	0.086	0.394	0.782	3.910	7.818	39.090	78.180	390.900	781.800

Graphical Comparison

Linear Scale



Logarithmic Scale



5. Analysis & Conclusion

Is GBN always faster? GBN is significantly faster in low-loss environments because pipelining allows multiple unacknowledged packets in flight.

Impact of increasing packet loss: Under high loss rates, GBN discards valid out-of-order packets and retransmits the entire window upon timeout, reducing its efficiency advantage.

Simplicity vs Efficiency: Stop-and-Wait is simple and requires no window management, making it suitable for resource-constrained systems. GBN offers higher channel utilization at the expense of window tracking and potential whole-window retransmissions.

Where Stop-and-Wait is useful: Stop-and-Wait is useful because its simple design ensures reliable data delivery with low implementation complexity, making it useful for low-bandwidth or low-latency networks where efficiency is less critical than correctness.